

Consolidação Técnica — Atividade Prática

N1-1

Fundamentos da Linguagem Java: Tipos Primitivos, Variáveis, Operadores e Expressões
(versão com entrada via teclado)

Disciplina: Linguagem de Programação I

Curso: ADS

Professor: Veríssimo

Semestre: 2026.2

≡ ORIENTAÇÕES GERAIS DA ATIVIDADE

- Todo o código-fonte desenvolvido para os 10 exercícios **deverá ser implementado em linguagem Java**, compilável e executável via linha de comando (`javac` / `java`).
- Cada exercício deverá ser entregue em um **arquivo `.java` individual**, seguindo rigorosamente a nomenclatura abaixo.
- Todos os arquivos gerados deverão ser adicionados à pasta do seu repositório GitHub **criado especificamente para esta disciplina** (conforme orientado em aula), dentro do diretório:
`LPI-Atividade-N1-1`
- Mantenha o nome da classe pública de cada arquivo Java coerente com o nome do arquivo (ex.: arquivo `LP-Atividade-N1-1-1234567890123.java` deve conter a classe pública correspondente).
- Comente o código sempre que uma decisão de projeto (uso de `final`, `BigDecimal`, curto-circuito, etc.) não for autoexplicativa.
- **Todos os programas deverão obter seus dados de entrada exclusivamente via teclado**, utilizando `java.util.Scanner`. Não é permitido fixar (hardcode) os valores de entrada diretamente no código — apenas os cálculos e a lógica devem estar fixos.
- **Ordem e formato das leituras:** siga exatamente a ordem de entrada especificada no quadro "Entradas via Teclado" de cada exercício. O testador automatizado fornece os valores pela entrada padrão (stdin) na ordem indicada, um valor por linha, e não interage manualmente com prompts.
- **Prompts:** utilize `System.out.print(...)` (sem quebra de linha) para exibir a mensagem de solicitação de cada dado antes de cada leitura, exatamente com o

texto indicado no quadro de entradas, para que a saída completa (prompt + eco, quando houver) corresponda ao padrão esperado.

- **Atenção:** A correção será automatizada. O testador alimentará a entrada padrão com os valores do quadro "Entrada de Teste" e comparará a saída padrão do seu programa, caractere a caractere, com a saída esperada. Divergências de texto, pontuação ou quebras de linha serão consideradas erro.

LP-Atividade-N1-1-nn-rrrrrrrrrrrrrrrrrr.java

LP-Atividade-N1-1 → prefixo fixo | **nn** → número do exercício dentro da atividade (01 a 10) | **rrrrrrrrrrrrrrrrrrrr** → número do seu RA, com 13 dígitos | **.java** → extensão do arquivo

Exemplo para o exercício 3, RA 1234567890123: LP-Atividade-N1-1-03-1234567890123.java

AVISO IMPORTANTE

Respostas extraídas de IA anularão este instrumento: NOTA ZERO.

Você **poderá** realizar pesquisas e consultas em material didático (**indicando as referências utilizadas**), mas **não poderá** consultar Inteligência Artificial (IA), nem inserir respostas provenientes de IA — total ou parcialmente.

Exercícios Práticos (10)

01 Tipos Primitivos: Declaração e Operações Básicas

TIPOS PRIMITIVOS

Implemente um programa que **leia via teclado** os dados de um aluno e os armazene nas 8 variáveis de tipos primitivos do Java (byte, short, int, long, float, double, char, boolean).

- Utilize **nomes de variáveis significativos**, compatíveis com o tipo primitivo de cada dado solicitado.
- Imprima todos os valores lidos utilizando `System.out.println`, um por linha, com rótulo descritivo, exatamente no formato do quadro de saída.
- Adicione um comentário indicando o tamanho em bits e o intervalo de valores de cada tipo utilizado.

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
---	-----------------	------	-------------------

1	Idade do Aluno:	byte	idade
2	Número de Faltas:	short	faltas
3	Matrícula ID:	int	matriculaId
4	Código Nacional do Estudante:	long	codigoNacional
5	Nota do Trabalho:	float	notaTrabalho
6	Nota da Prova Final:	double	notaProvaFinal
7	Conceito Final do Aluno:	char	conceitoFinal
8	Aluno está Aprovado (true/false):	boolean	aprovado

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

```
Idade do Aluno: 21
Número de Faltas: 4
Matrícula ID: 987654
Código Nacional do Estudante: 8839201938472
Nota do Trabalho: 8.5
Nota da Prova Final: 9.25
Conceito Final do Aluno: A
Aluno está Aprovado (true/false): true

--- Dados do Aluno Fictício ---

Idade do Aluno: 21 anos
Número de Faltas: 4
Matrícula ID: 987654
Código Nacional do Estudante: 8839201938472
Nota do Trabalho: 8.5
Nota da Prova Final: 9.25
Conceito Final do Aluno: A
Aluno está Aprovado? true
```

Arquivo: LP-Atividade-01-01-rrrrrrrrrrrrrrr.java

02 Identidade vs. Valor: Wrappers e o Operador ==

WRAPPERS

Implemente um programa que **leia via teclado** os valores necessários para demonstrar, na prática, a diferença entre comparação de **valor** e comparação de **identidade** em Java (Item 61, Joshua Bloch).

- Leia um valor `int` e utilize-o para inicializar dois primitivos iguais, comparando-os com `==`.
- Leia um segundo valor `int` e crie dois objetos `Integer` via `new Integer(...)` com esse mesmo valor, comparando-os com `==` e com `.equals()`.
- Leia um terceiro valor (esperado entre -128 e 127) e um quarto valor (esperado fora desse intervalo) para demonstrar o comportamento do *Integer Cache* com autoboxing, comparando cada par com `==`.
- Imprima, para cada caso, uma mensagem clara indicando o resultado, calculado dinamicamente a partir dos valores lidos (não fixe os resultados no código).

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Valor para comparação de primitivos:	int	valorPrimitivo
2	Valor para comparação de objetos (new Integer):	int	valorObjeto
3	Valor dentro do Integer Cache (-128 a 127):	int	valorCache
4	Valor fora do Integer Cache:	int	valorForaCache

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

```
Valor para comparação de primitivos: 10
Valor para comparação de objetos (new Integer): 10
Valor dentro do Integer Cache (-128 a 127): 120
Valor fora do Integer Cache: 200

--- Comparação de Primitivos (int) ---
aPrimitivo == bPrimitivo: true

--- Comparação de Objetos via 'new Integer()' ---
aObjeto == bObjeto (Identidade): false
aObjeto.equals(bObjeto) (Valor): true

--- Comparação com Autoboxing e Integer Cache ---
Dentro do Cache (120) -> xCache == yCache: true
Fora do Cache (200) -> xForaCache == yForaCache: false
Fora do Cache (200) -> xForaCache.equals(yForaCache): true
```

Arquivo: LP-Atividade-01-02-rrrrrrrrrrrrrrrr.java

03 Tratamento de NullPointerException por Auto-unboxing

WRAPPERS

Implemente um programa que simule o cadastro da idade de um usuário utilizando um objeto Integer que inicialmente não foi preenchido (null), e que em seguida **leia via teclado** um valor válido para reatribuição.

- Declare uma variável Integer `idade = null;` (sem qualquer leitura nesta etapa).
- Tente realizar uma operação aritmética que force o *unboxing* automático dessa variável.
- Capture a exceção gerada com um bloco `try/catch`, exibindo uma mensagem amigável ao usuário.
- Em seguida, **leia via teclado** um novo valor de idade, reatribua-o à variável e demonstre que a mesma operação funciona corretamente.

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Informe uma idade válida:	int	novaldade

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

Tentando realizar operação aritmética com Integer nulo...

```
Erro Capturado com sucesso: Não foi possível calcular porque a idade não foi informada (null).
```

Detalhe da exceção: java.lang.NullPointerException

Informe uma idade válida: 25

Reatribuindo valor válido para a variável...

Operação bem-sucedida! Idade atual: 25 | Idade no próximo ano: 26

[illegible]

04 Variáveis final e Constantes de Classe

VARIÁVEIS / CONSTANTES

Implemente uma classe que represente as configurações de um pequeno sistema de biblioteca, aplicando os princípios de **escopo mínimo** e **minimização da mutabilidade** defendidos por Joshua Bloch, lendo via teclado os dados variáveis do empréstimo.

- Declare ao menos duas constantes de classe (`public static final`), como o nome da instituição e o prazo máximo de empréstimo em dias (mantenha estes valores fixos no

código, pois são constantes institucionais).

- **Leia via teclado** o número de dias que o livro ficará emprestado e passe esse valor como parâmetro `final` para um método que calcule/valide a data limite de devolução.
- Declare variáveis locais utilizando `final` sempre que seu valor não deva ser alterado após a inicialização.
- Inclua, como comentário, uma tentativa de reatribuição a uma variável `final` demonstrando o erro de compilação (sem deixar a linha ativa no código).

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Informe o número de dias do empréstimo:	int	diasEmprestimo

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

Informe o número de dias do empréstimo: 10

Instituição: FATEC Ipiranga

Prazo Máximo Padrão: 14 dias.

Dias calculados para devolução: 10

[illegible]

05

Operadores Aritméticos, Relacionais e de Atribuição Composta

OPERADORES

Implemente um programa de controle de estoque simples, **lendo via teclado** o estoque inicial, as movimentações e os parâmetros de verificação.

- Leia o estoque inicial e as três movimentações (entrada e saídas), aplicando-as com operadores de atribuição composta (+=, -=) na ordem em que forem lidas.
- Leia o estoque mínimo e utilize operadores relacionais para determinar se o estoque final está abaixo desse limite.
- Leia o tamanho do lote (caixa) e utilize o operador % para verificar se a quantidade final em estoque é divisível por esse valor.
- Exiba mensagens claras para cada verificação realizada, com os valores calculados dinamicamente.

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
---	-----------------	------	-------------------

1	Estoque inicial:	int	estoque
2	Quantidade de entrada:	int	qtdEntrada
3	Quantidade de saída 1:	int	qtdSaida1
4	Quantidade de saída 2:	int	qtdSaida2
5	Estoque mínimo:	int	estoqueMinimo
6	Tamanho do lote (caixa):	int	tamanhoLote

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

Estoque inicial: 50

Quantidade de entrada: 22

Quantidade de saída 1: 40

Quantidade de saída 2: **18**

Estoque mínimo: 15

Tamanho do lote (caixa): 12

Estoque inicial: 50 unidades.

Após entrada (+22): 72 unidades.

Após saída (-40): 32 unidades.

Após outra saída (-18): 14 unidades.

```
0 nível de estoque atual (14) está abaixo do mínimo (15)? true
```

Unidades fora de caixas fechadas de 12: 2

O estoque está perfeitamente fracionado em caixas completas? false

Arquivo: LP-Atividade-01-05-rrrrrrrrrrrrrrrr.java

06 Curto-Circuito Lógico com Validação de Texto

OPERADORES LÓGICOS

Implemente um programa que **leia via teclado** um texto de entrada (por exemplo, um nome de usuário), que pode vir vazio, explorando o comportamento de curto-circuito dos operadores lógicos.

- Leia uma linha de texto com `Scanner.nextLine()` e armazene em uma `String` `texto` (o testador executará o programa três vezes: com um texto válido, com uma linha vazia, e simulando ausência de valor).
- Utilize o operador `&&` de curto-circuito para verificar, de forma segura, se o texto não é nulo e possui comprimento maior que zero.
- Em um comentário, explique por que o uso do operador `&` simples nesse mesmo contexto poderia lançar uma `NullPointerException` caso a referência fosse nula.

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Informe o nome de usuário:	String (linha)	texto

 SESSÕES DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

--- Execução 1 ---

Informe o nome de usuário: **usuario_fatec**

Usuário válido fornecido: usuario_fatec

--- Execução 2 ---

Informe o nome de usuário:

Entrada rejeitada: O texto está nulo ou vazio.

Arquivo: LP-Atividade-01-06-rrrrrrrrrrrrrrrr.java

07 Precisão Decimal com BigDecimal

PRECISÃO NUMÉRICA

Implemente uma calculadora de valores financeiros que corrija o problema de imprecisão do padrão IEEE 754 (Item 48, Joshua Bloch), **lendo via teclado** os valores utilizados na demonstração e no cálculo de parcelas.

- Leia dois valores decimais (como `String`) e reproduza o erro de precisão subtraindo-os como `double`, imprimindo o resultado impreciso.
- Refaça o mesmo cálculo utilizando `java.math.BigDecimal`, instanciando os valores **sempre via `String`** (reaproveitando as mesmas entradas lidas).
- Leia o valor total de uma compra e o número de parcelas, e implemente a divisão utilizando `divide(...)` com escala de 2 casas decimais e `RoundingMode.HALF_UP`.
- Compare, via impressão no console, o resultado impreciso e o resultado exato.

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Informe o valor A (ex.: 1.00):	String	valorA
2	Informe o valor B (ex.: 0.90):	String	valorB
3	Informe o valor total da compra:	String	valorCompra
4	Informe o número de parcelas:	int	numeroParcelas

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

```
Informe o valor A (ex.: 1.00): 1.00
Informe o valor B (ex.: 0.90): 0.90
Informe o valor total da compra: 100.00
Informe o número de parcelas: 3
--- Demonstração da Imprecisão do padrão IEEE 754 (double) ---
Resultado esperado de 1.00 - 0.90 seria 0.10
Resultado real obtido com double: 0.09999999999999998
--- Correção exata utilizando java.math.BigDecimal ---
Resultado com BigDecimal (String Constructor): 0.10
--- Divisão de parcelas com Escala e RoundingMode.HALF_UP ---
Compra de R$ 100.00 dividida em 3x: R$ 33.33 por parcela.
```

Arquivo: LP-Atividade-01-07-rrrrrrrrrrrrrrr.java

08 Abordagem Escalar com Inteiros (Centavos)

PRECISÃO NUMÉRICA

Implemente a mesma calculadora de valores financeiros do exercício anterior, agora utilizando a estratégia de **mapeamento escalar com inteiros** (valores em centavos em vez de reais), lendo os dados via teclado.

- Leia o valor total da compra, em reais, como `String` ou `double`, e converta-o para `long` em centavos.
- Leia o número de parcelas desejado.
- Realize as operações aritméticas necessárias estritamente sobre os valores inteiros (centavos).
- Converta o resultado final para `double` **apenas no momento da exibição** ao usuário.
- Compare, em comentário, as vantagens e desvantagens dessa abordagem em relação ao uso do `BigDecimal` do exercício 07.

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Informe o valor total da compra em reais:	double	valorTotalReais
2	Informe o número de parcelas:	int	numeroParcelas

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

Informe o valor total da compra em reais: 100.00

Informe o número de parcelas: 3

--- Calculadora Financeira Escalar (Mapeamento em Centavos) ---

Valor total convertido: 10000 centavos.

Divisão de R\$100.00 por 3 em centavos: 3333 centavos por parcela.

Valor convertido para exibição: R\$ 33.33

Arquivo: LP-Atividade-01-08-rrrrrrrrrrrrrrrr.java

09

Operador Ternário: Simplificação de Estruturas Condicionais

EXPRESSÕES

Implemente um programa que **leia via teclado** a nota final de um aluno e determine sua situação (Aprovado/Reprovado), comparando duas abordagens de implementação.

- Leia a nota final do aluno.
- Implemente primeiro a lógica de aprovação/reprovação ($\text{nota} \geq 6$) utilizando uma estrutura `if / else` tradicional.
- Em seguida, reimplente a **mesma lógica** utilizando o operador ternário `? :`.
- No código, inclua um comentário demonstrando como ficaria um ternário **encadeado** (nested ternary) caso houvesse um terceiro estado (ex.: Exame), e explique por que essa forma **deve ser evitada**.

ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Informe a nota final do aluno:	double	notaFinal

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

Informe a nota final do aluno: 7.0

```
--- Abordagem 1: Estrutura Condicional Tradicional (if-else) ---
```

Situação (if-else): Aprovado

```
--- Abordagem 2: Operador Ternário Simples ---
```

Situação (Ternário): Aprovado

Arquivo: LP-Atividade-01-09-rrrrrrrrrrrrrrr.java

10

Precedência de Operadores e Expressão Lógica Combinada

EXPRESSÕES

Implemente um programa que **leia via teclado** os operandos de uma expressão aritmética e os dados de um aluno (média e presença), avaliando o critério de aprovação direta e evidenciando o entendimento da precedência de operadores.

- Leia quatro valores numéricos e monte uma expressão aritmética composta (mínimo 4 operadores), calculando-a primeiro **sem parênteses** e depois **com parênteses explícitos**, comprovando que o resultado é o mesmo, porém a segunda forma é mais legível.
- Leia a média e o percentual de presença do aluno e implemente a expressão lógica `(media >= 6) && (presenca >= 75)` para determinar a variável booleana `aprovadoDireto`.
- O testador executará o programa três vezes, com conjuntos de valores diferentes, reproduzindo as três linhas da tabela-verdade apresentada em aula.
- Imprima, em cada execução, os valores de entrada e o resultado da avaliação.

 ENTRADAS VIA TECLADO (NESTA ORDEM)

#	PROMPT A EXIBIR	TIPO	VARIÁVEL SUGERIDA
1	Informe o valor A:	double	a
2	Informe o valor B:	double	b
3	Informe o valor C:	double	c
4	Informe o valor D:	double	d
5	Informe a média do aluno:	double	media
6	Informe o percentual de presença:	double	presenca

 SESSÃO DE TESTE (ENTRADA EM AZUL / SAÍDA EM VERDE):

```
Informe o valor A: 10
Informe o valor B: 5
Informe o valor C: 2
Informe o valor D: 2.5
Informe a média do aluno: 7.5
Informe o percentual de presença: 80
--- Demonstração de Precedência Aritmética ---
Resultado Sem parênteses: 12.5
Resultado Com parênteses explícitos: 12.5
Nota: Ambos dão o mesmo resultado pela precedência natural (*, /
```

depois +), mas a segunda forma é mais legível.

--- Validação do Critério de Aprovação ---

Entrada -> Média: 7.5 | Presença: 80%

Resultado da avaliação (aprovadoDireto): true

Arquivo: LP-Atividade-01-10-rrrrrrrrrrrrrrr.java

FATEC Ipiranga — Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas (ADS)

Linguagem de Programação I — Prof. Veríssimo — 2026.2 -carlos.pereira01@cps.sp.gov.br

Material didático de apoio exclusivo. Proibida a reprodução sem autorização prévia.